

《云计算与大数据处理》

实验报告

专　　业：计算机科学与技术
班　　级：24 计科 3
学　　号：2410311308
姓　　名：龚治榕
实验（一）：云端数据接口与可复现环境
实验日期：2026.10.20

《云计算与大数据处理》课程实验报告

实验名称	云端数据接口与可复现环境					实验序号	01
姓　　名	龚治榕	专　业	计科	班　　级	24 计科 3	学　　号	2410311308
实验类型	综合型	实验日期	10.20	指导教师	刘锦行	成　　绩	

一、实验目的与环境

实验目的：

1.区分本地文件路径、对象键、HTTP endpoint 与内容哈希四个概念，并用 SHA-256 独立核对一次真实往返。

2.查看一条合成服务调用事件，说明接口请求成功与数据业务合法是两种不同的检查。

3.在脚手架的个人副本中修改对象键规则，用一个变式验证修改效果。

4.说明本地 Moto 模拟服务的能力边界，不把模拟成功写成生产保证。

5.主动注入一次连接故障，依据错误信息定位原因并在修复后重新取证。

对象存储：按“桶 + 对象键”保存完整数据对象的服务，常见用途是日志、模型文件、图片和数据集。对象键可以含斜杠，但不等于传统文件系统中的目录。S3 API：一组常见的对象存储 HTTP 接口，课程用 Moto 在本机启动服务来模拟。内容哈希：本实验用 SHA-256 把任意长度字节序列映射为固定长度摘要，以两份文件摘要相同作为内容一致的工程证据。

实验环境：硬件与系统基线

Windows 10.0.26200（64 位），CPU 运行，Python 3.12.10，八次实验共用一个虚拟环境

关键依赖版本(requirements.lock.txt 核对 99 个包全部一致)

boto3 1.43.93、moto 5.2.3、pandas 3.0.5、duckdb 1.5.5、dask 2026.8.0、fastapi 0.141.1

本实验关键配置

课程根目录 E:\Desktop\2410311308_龚治榕_云计算实验\BDP，解释器 .venv\Scripts\python.exe，对象存储为 127.0.0.1 上临时启动的 Moto S3 服务，测试凭据 testing 仅用于本机模拟

代码、截图与运行证据仓库：https://github.com/yiyun2026/bdp-coursework（点击可跳转）

任务：把 20,102 行合成事件经 S3 接口上传再下载，用 SHA-256 核对内容一致，并完成对象键规则的个人修改与故障恢复

二、实验内容与步骤

工作目录:2410311308_龚治榕_云计算实验\BDP(课程仓库根目录，个人修改在 submissions\lab01)

o 实验内容

本实验围绕“模拟服务使用分析与规则查询平台”的数据接入环节，练习对象存储往返，检验一份文件经过对象接口传输后内容是否仍然保持原样。先用固定随机种子生成合成事件并保存为本地原始文件，再通过 S3 接口上传到本机 Moto 服务、下载为另一个文件，最后比较两份内容哈希，并把原始数据交给实验 02 继续处理。

本章实际链路如下：

固定随机种子生成合成事件 → 保存本地原始文件 → 按 S3 API 上传到本机 Moto HTTP

服务 → 下载为另一个文件 → 比较内容哈希 → 将原始数据交给实验 02
对象键：桶内对象的名称，本实验按“bronze/events/”加源文件 SHA-256 再加 .jsonl 构造，属于内容寻址方式，相同内容得到相同键。版本 ID：桶启用版本后，向同一键写入两次相同内容会产生两个不同的 VersionId，它标识一次对象版本，与标识字节内容的摘要用途不同。

o 实验步骤

1. Windows 环境与独立虚拟环境

1.1 设置编码并确认课程根目录

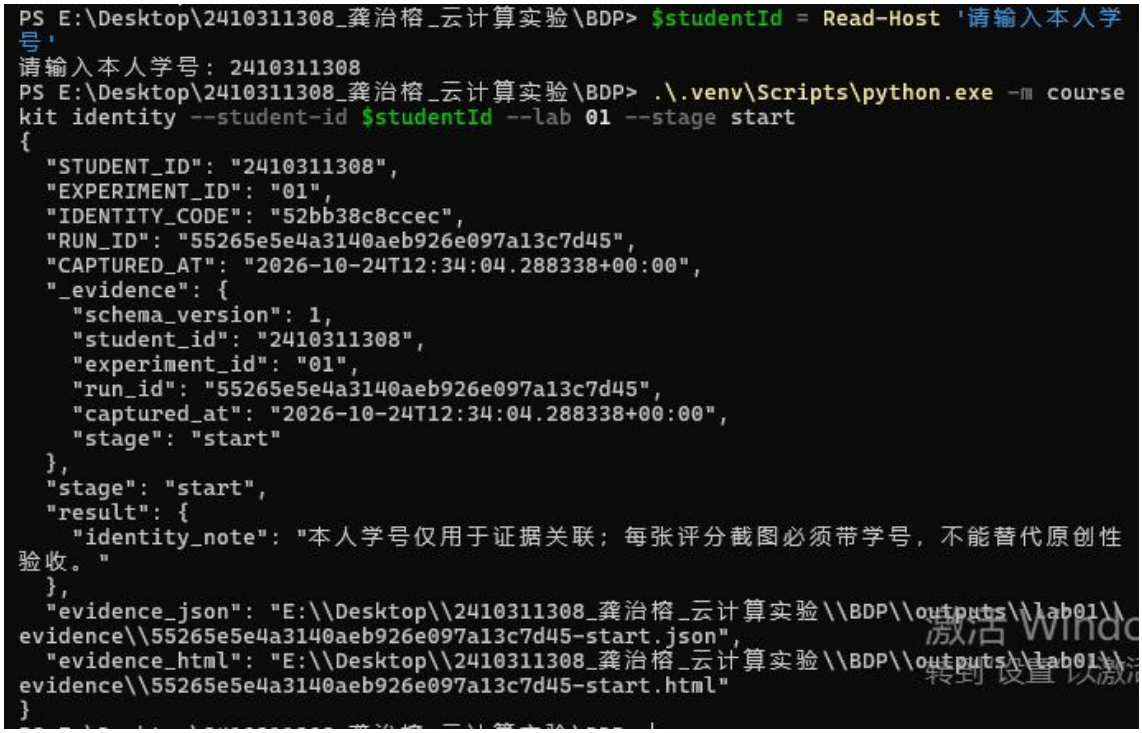
在课程根目录打开 PowerShell，设置 PYTHONUTF8 让新启动的 Python 使用 UTF-8，避免中文路径造成乱码；该设置只影响当前窗口，新窗口需要重设。

```
PS E:\Desktop\2410311308_龚治榕_云计算实验\BDP> cd E:\Desktop\2410311308_龚治榕_云计算实验\
PS E:\Desktop\2410311308_龚治榕_云计算实验\BDP> $env:PYTHONUTF8 = '1'
PS E:\Desktop\2410311308_龚治榕_云计算实验\BDP> Get-ChildItem -Name
.git
.pytest_cache
.venv
00. 课程导论与准备
01. 云端数据接口与可复现环境
02. 数据湖与质量契约
03. 分布式批处理与性能分析
04. 增量事件处理与故障恢复
05. AI知识数据与混合检索
06. 证据问答与系统评价
07. 云服务部署与推理可观测性
08. 数据治理与可信发布
coursekit
data
docs
outputs
scripts
submissions
tests
报告模板
控制文档
.dockerignore
.gitattributes
.gitignore
compose.yaml
Dockerfile
README.md
requirements-models.lock.txt
requirements.container.txt
requirements.lock.txt
requirements.txt
student.json
交付文件清单.json
PS E:\Desktop\2410311308_龚治榕_云计算实验\BDP> py -3.12 --version
Python 3.12.10
PS E:\Desktop\2410311308_龚治榕_云计算实验\BDP> .\.venv\Scripts\python.exe -m pip check
No broken requirements found.
PS E:\Desktop\2410311308_龚治榕_云计算实验\BDP>
```

Python 3.12.10，pip check 返回 No broken requirements found

1.2 登记本人学号

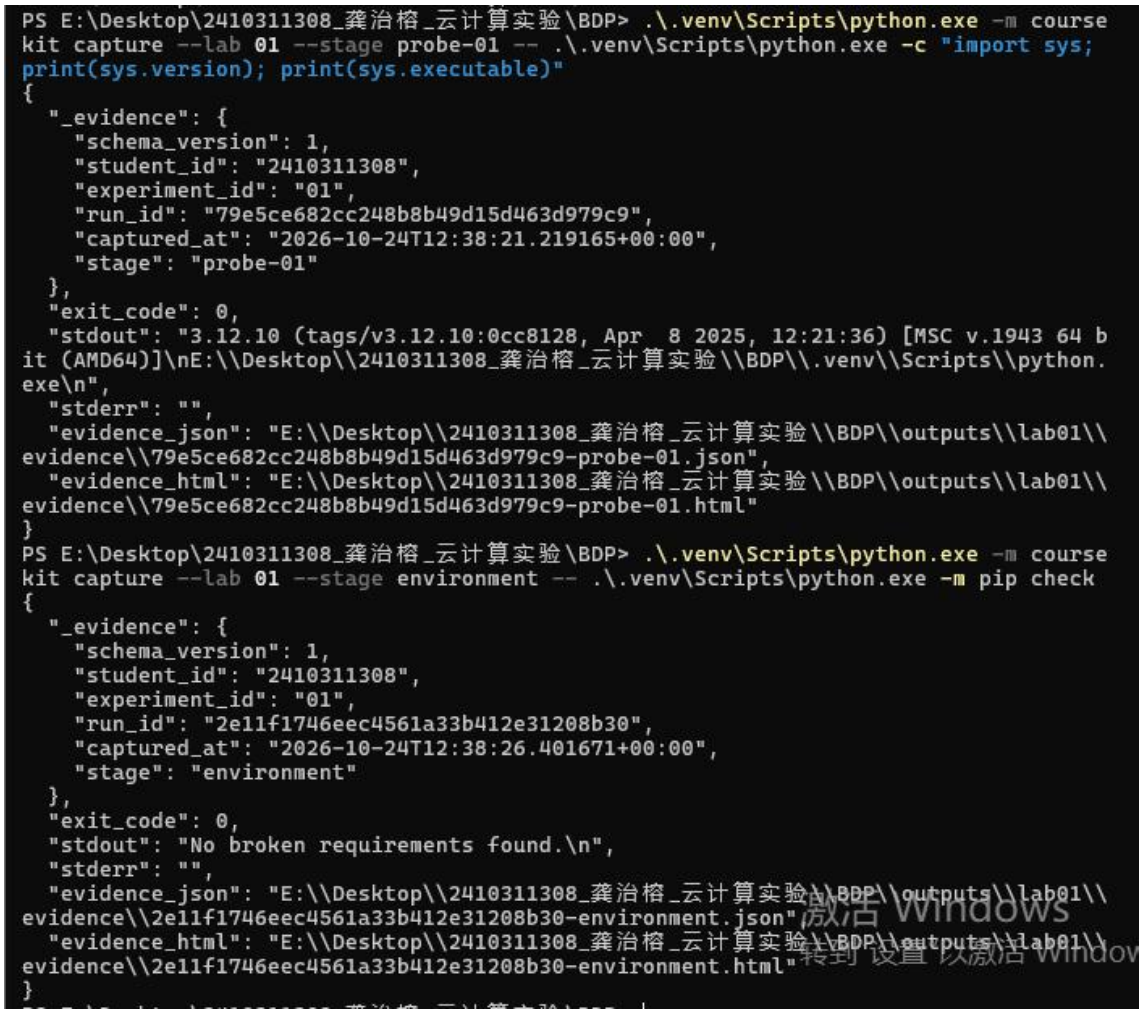
身份登记只建立本人标识，不能代替实验结果。登记后输出的 IDENTITY_CODE 由学号与实验号决定，同一学号同一实验号下稳定不变。



学号 2410311308，实验号 01，IDENTITY_CODE = 52bb38c8ccec

1.3 采集环境证据

用 capture 包装实际命令，记录真实 stdout、stderr 与退出码，并生成带学号和 run_id 的证据文件。



probe-01 与 environment 两个 stage 均 exit_code = 0



probe-01 证据页：解释器为 .venv\Scripts\python.exe，Python 3.12.10

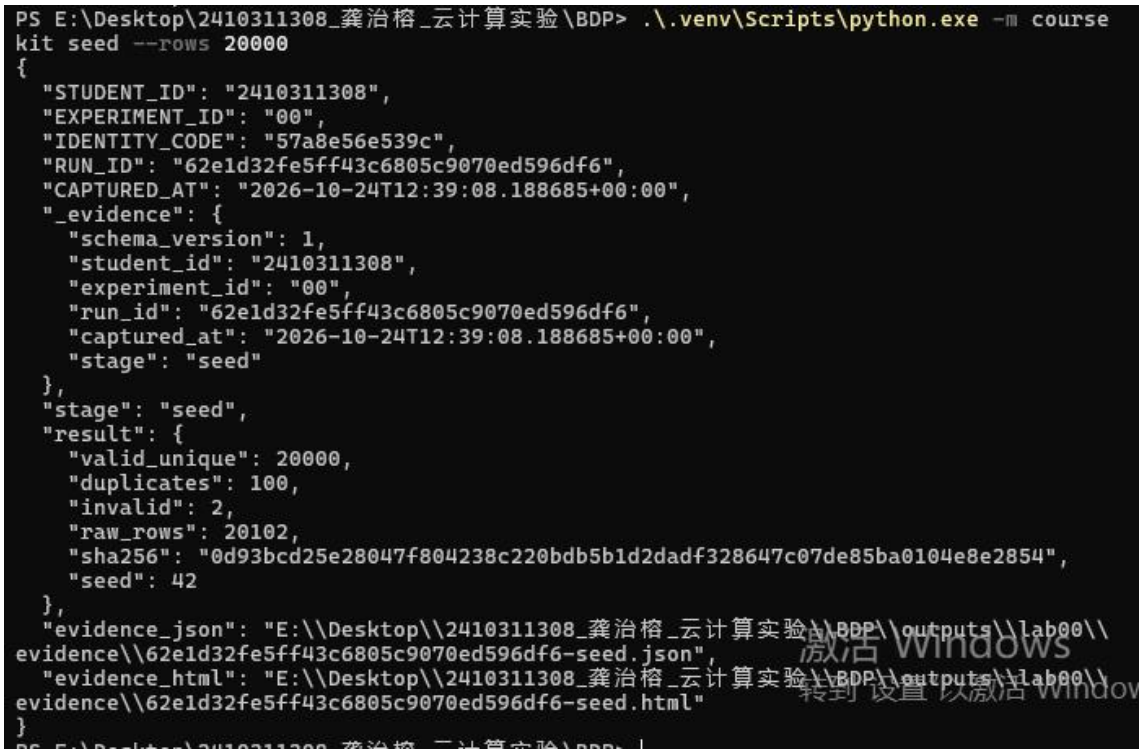


environment 证据页：No broken requirements found.

2. 生成原始数据并核对规模

2.1 运行 seed 生成数据

默认基线为 20,000 个合法独特事件、100 条完全重复事件和 2 条不合法事件。固定随机种子使相同版本的生成器和相同参数能够复现数据。



valid_unique=20000、duplicates=100、invalid=2、raw_rows=20102，seed=42

2.2 核对行数与首条事件字段

先统计行数，再打印第一条事件。输出应包含 event_id、event_time、service、model、input_tokens、output_tokens、latency_ms、status 八个字段。



lines= 20102，首条事件字段完整，event_time 带 +00:00 时区偏移



probe-02 证据页：行数 20102 与首条事件同屏

2.3 独立复核数据组成

本章只观察异常，不手工删除原始文件中的坏行。为保证脚手架报告的数字属实，另用独立脚本逐行解析复核：总行数 20,102 行，无可解析失败的行；100 个 event_id 各出现两次，对应 100 行完全重复内容；发现一条 input_tokens 为 -1 的事件（event_id 为 bad-negative）。复核结果与脚手架一致，原始文件未被改动。

3. 运行前的哈希预测

3.1 核心预测

手册要求先写出预测，再运行验证。我的预测是：同一份字节内容上传到两个不同的对象键，内容哈希不会改变；而只要改动 JSON 里的一个数字，哈希就会改变。
预测理由是：SHA-256 的计算输入只有对象 Body 的字节序列。桶名、对象键和 endpoint 属于 HTTP 请求的主机与路径部分，只用于定位对象，不进入摘要计算；服务端在下载时也不会对内容重新编码。因此只要字节相同，摘要必然相同，与它放在哪个键下无关；反过来，只要字节里有一个数字变了，哪怕只差一个字符，摘要也会雪崩式地完全不同。
3.2 三项对照的预测与实测
除第一项必做预测外，另做两项对照。三项的运行前预测与实测结果如下：
相同字节上传到两个不同对象键：预测哈希不变，实测两键均为 23c0b20c…f0e8，与预测一致。
只改变 JSON 缩进，数据值不变：预测哈希改变，实测 23c0b20c…f0e8 变为 de27ead6…a195，解析后数据值相等，与预测一致。
将 JSON 中一个数字改为另一个数字：预测哈希改变，实测 23c0b20c…f0e8 变为 5657a58e…d3f6，与预测一致。
第二项最值得注意：只调整缩进后两个文件的 JSON 语义完全相同，解析出来的字典相等，但字节和摘要都变了。这说明字节一致性与语义一致性是两个不同层面的检查；反过来，摘要一致也不能证明事件含义正确，一条 input_tokens 为 -1 的记录，摘要同样可以稳定一致。
3.3 云 API 模拟的能力边界
本章使用的是 Moto 在本机启动的模拟服务，不是真实云存储。为避免夸大结论，逐项说明如下。
本地 HTTP 请求真实发出：本实验验证，依据是访问日志出现 PUT、HEAD、GET 请求。
S3 接口语义：本实验验证，依据是建桶、开启版本、两次 put_object、download_file 均返回 200。
字节层面内容一致：本实验验证，依据是源文件与下载副本 SHA-256 相同。
事件业务合法：本实验未验证，依据是原始数据仍含 2 条不合法事件，本章不处理。
云存储耐久性：本实验未验证，依据是服务为内存模拟，进程退出后对象消失。
跨机容灾：本实验未验证，依据是仅单机回环地址，无副本与故障切换。
真实访问控制 / IAM：本实验未验证，依据是测试凭据 testing 仅用于本机模拟，无真实权限。
未验证的四项并不等于实验失败，而是本章的观察范围本就不包括它们。因此本章完成的是“本地云 API 模拟与数据往返校验”，不能表述为“完成了云部署”。
4. 运行对象存储往返实验
4.1 运行主实验
程序自动启动临时 Moto HTTP 服务，创建桶并开启版本控制，通过 S3 客户端上传原始数据两次、下载一次，最后结束服务。终端结果与 outputs/lab01/storage.json 保存本次证据，下载副本为 outputs/lab01/downloaded-events.jsonl。



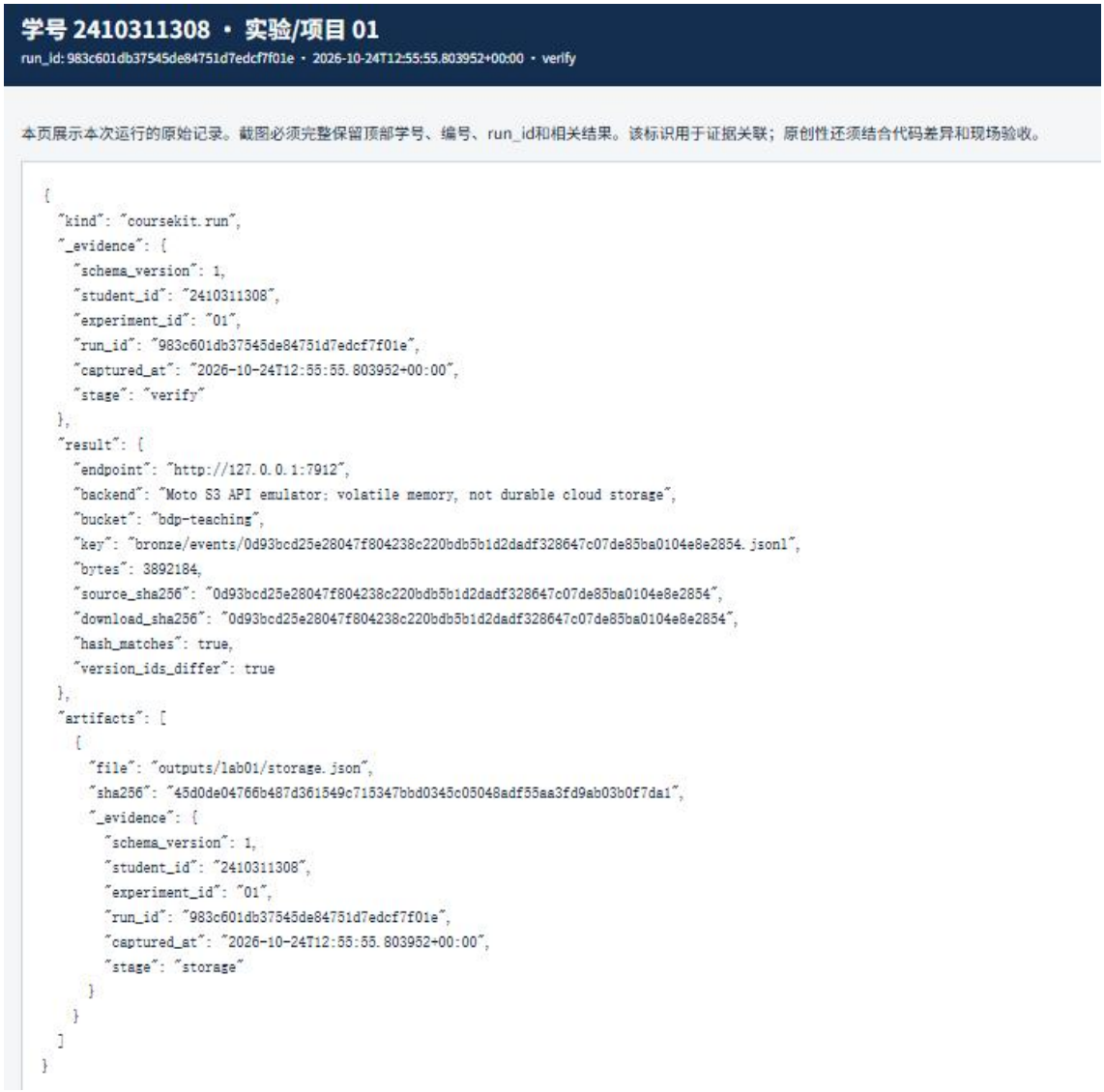
hash_matches = true、version_ids_differ = true，endpoint 与双哈希同屏

4.2 阅读核心代码的调用顺序

打开 coursekit/pipeline.py，实验 01 的实际操作顺序为：先用 create_bucket 创建逻辑容器，再 put_bucket_versioning 开启版本，然后以 'bronze/events/' 加源文件 SHA-256 再加 .jsonl 构造对象键，连续 put_object 两次写入相同字节，最后 download_file 写入另一个本地路径。对象键不是下载目标路径，下载目标路径是另一个本地文件。

4.3 独立复算哈希

为避免只依赖程序自己打印的判定，另写脚本直接读取 data/raw/events.jsonl 与 outputs/lab01/downloaded-events.jsonl 重新计算摘要，两者同为 0d93bcd25e28047f804238c220bdb5b1d2dadf328647c07de85ba0104e8e2854，字节数同为 3,892,184，相等判定为 True。这一步排除了判定逻辑写错却仍输出 true 的可能。

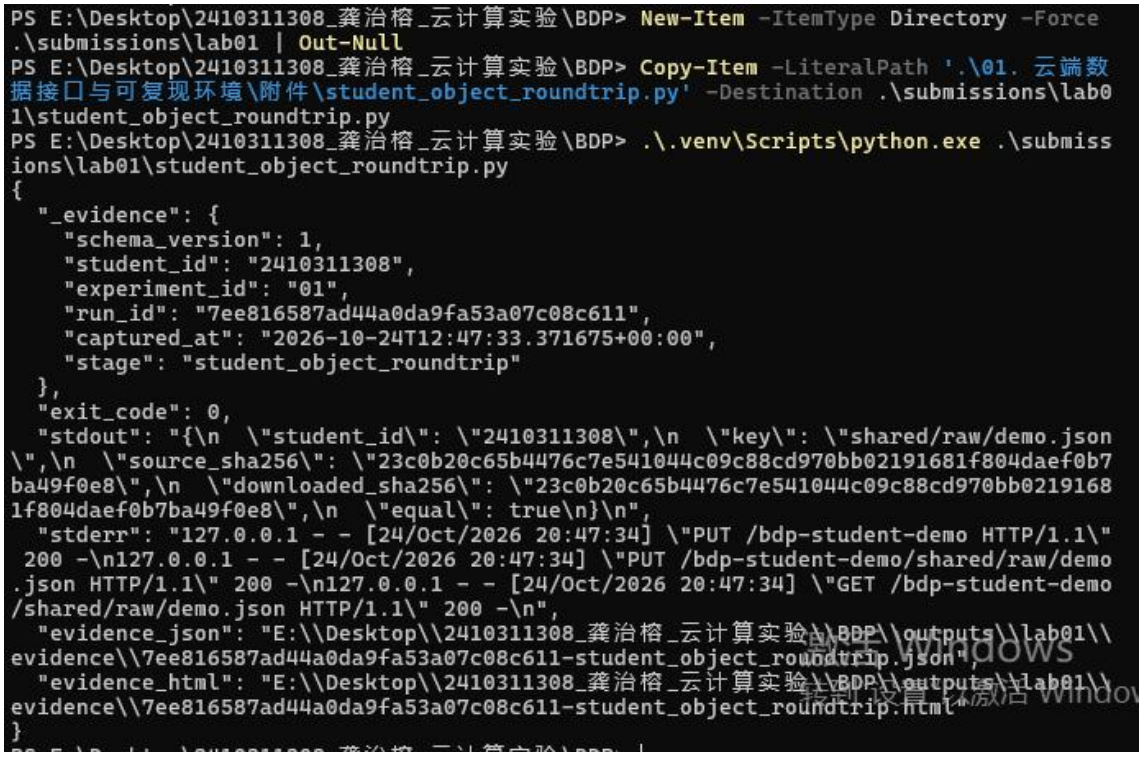


verify 证据页：桶、对象键、双哈希与两个布尔判定同屏

5. 个人修改：对象键规则与 payload 对照

5.1 保存修改前基线

将附件复制到 submissions/lab01 后先原样运行一次，附件自动读取已登记学号，不需要手改 STUDENT_ID 常量。



基线：对象键 shared/raw/demo.json，SHA-256 为 23c0b20c…f0e8，equal = true



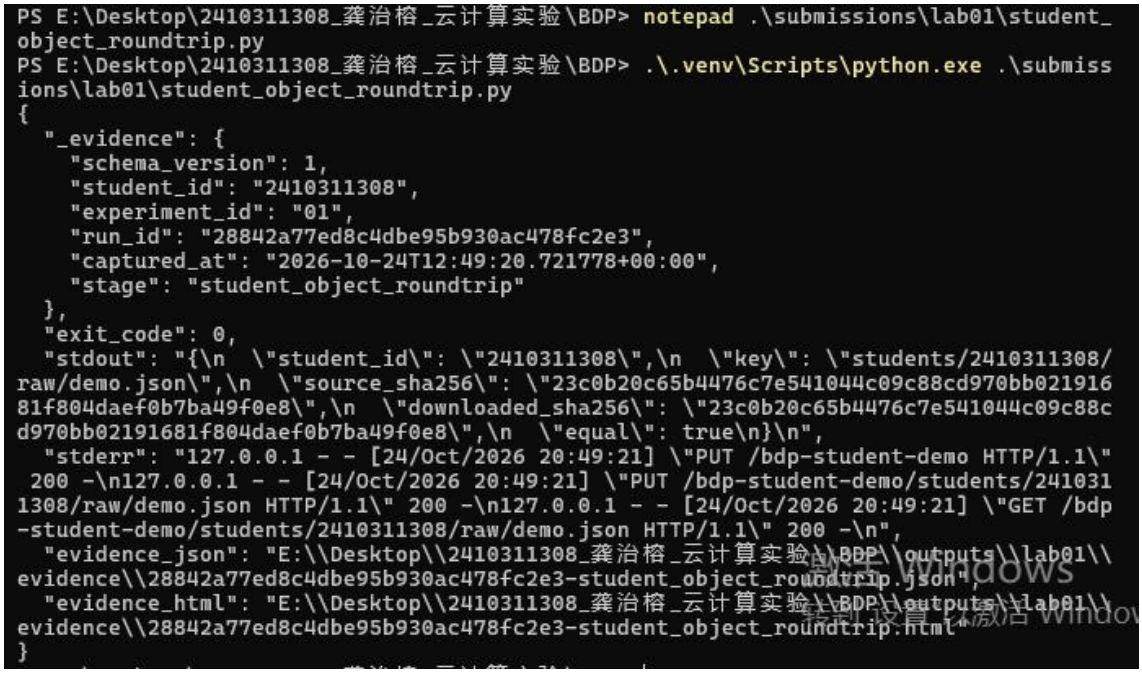
基线证据页，run_id 7ee816587ad44a0da9fa53a07c08c611

5.2 修改 make_key() 为含本人学号的前缀

只修改 make_key() 的返回值，不触碰 payload 与哈希判定。

```
return "students/2410311308/raw/demo.json"
```

修改理由：原规则所有同学写入同一个键 shared/raw/demo.json，后提交的人会覆盖先前对象；加入学号后每个提交者拥有自己的命名空间，不再互相覆盖。对象键中的斜杠只是键名的一部分，不代表文件系统目录，因此这种“学号目录”只是命名约定，隔离仍需访问控制实现。



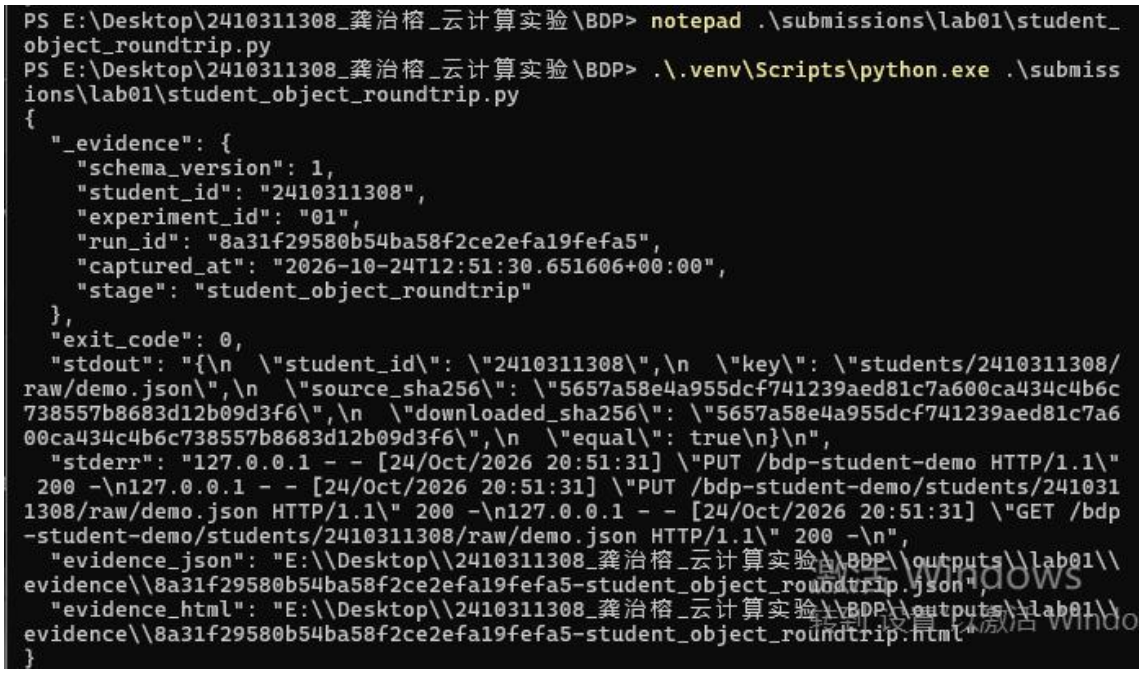
改键后：对象键变为 students/2410311308/raw/demo.json，SHA-256 仍为 23c0b20c…f0e8



改键证据页，run_id 28842a77ed8c4dbe95b930ac478fc2e3

5.3 修改 payload 的一个数值

在个人副本中把 payload 的 requests 由 3 改为 4，保持键与判定不变，再次运行。运行前预计：对象键不变，源与下载的 SHA-256 都会变成同一个新值，相等判定仍为 true。实测完全吻合。



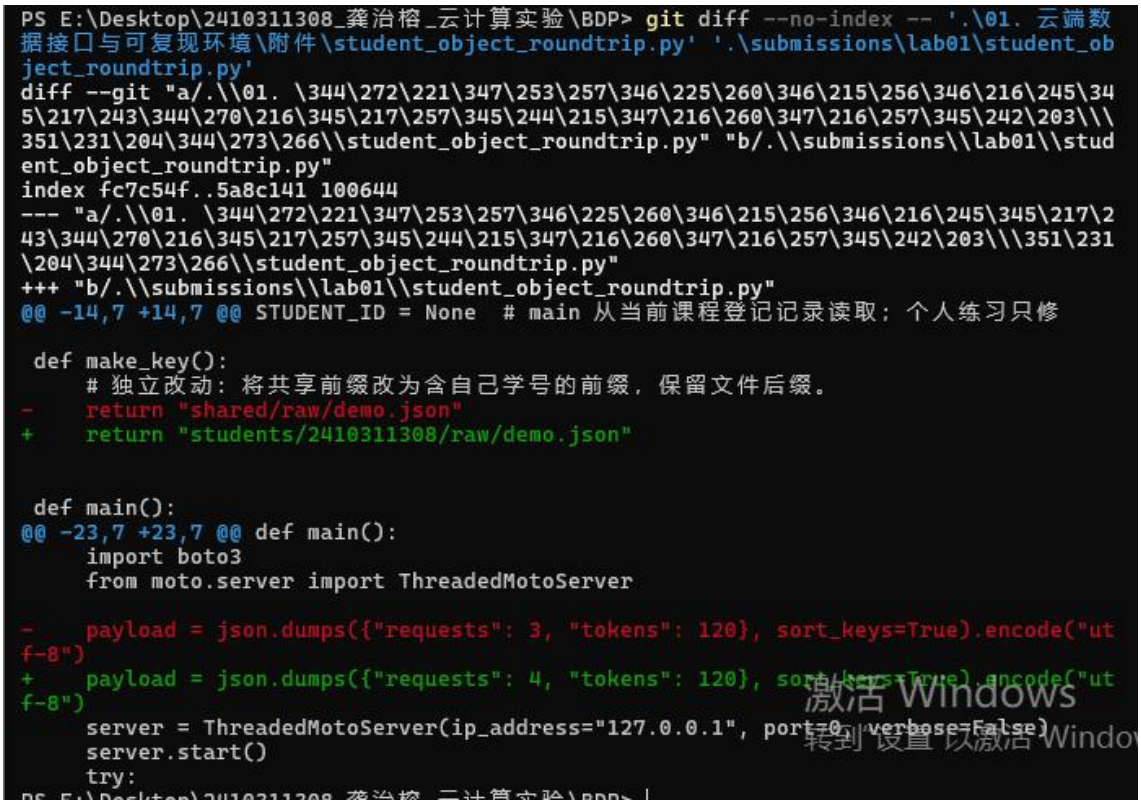
改 payload 后：SHA-256 变为 5657a58e…d3f6，equal 仍为 true



改 payload 证据页，run_id 8a31f29580b54ba58f2ce2efa19fefa5

5.4 生成并核对代码差异

用 git diff --no-index 比较附件与个人副本。Git 在存在差异时返回码为 1，这是正常比较结果，不是运行失败。

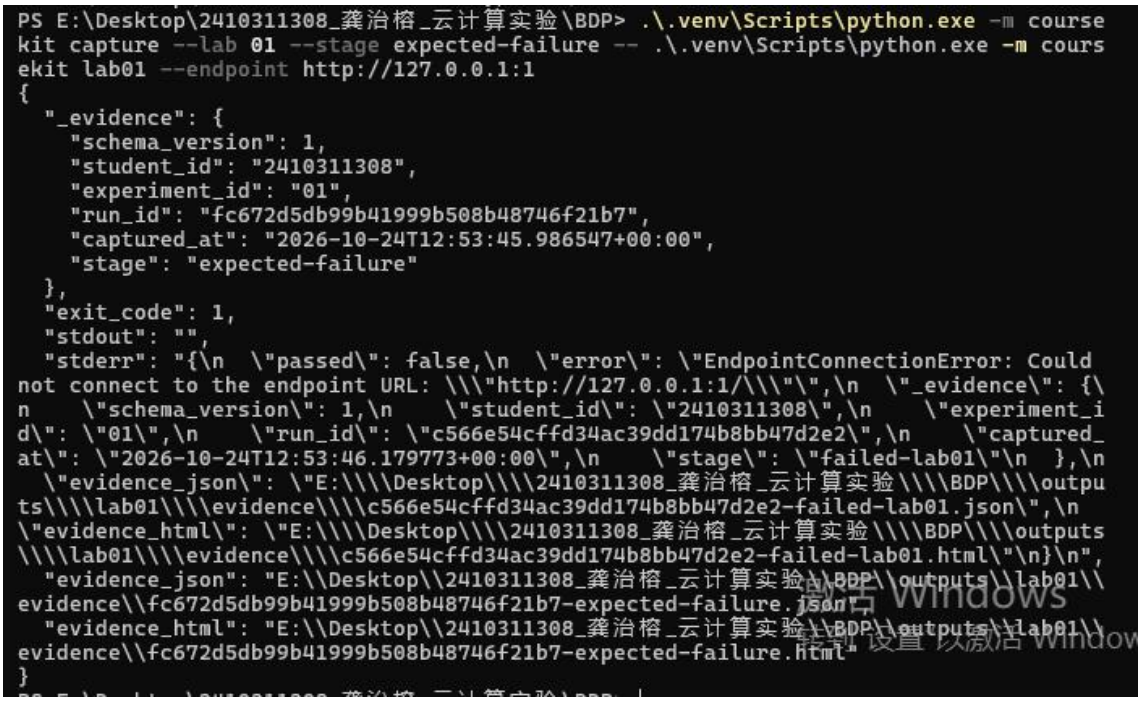


差异只有两处：make_key 的返回值与 payload 的 requests 数值

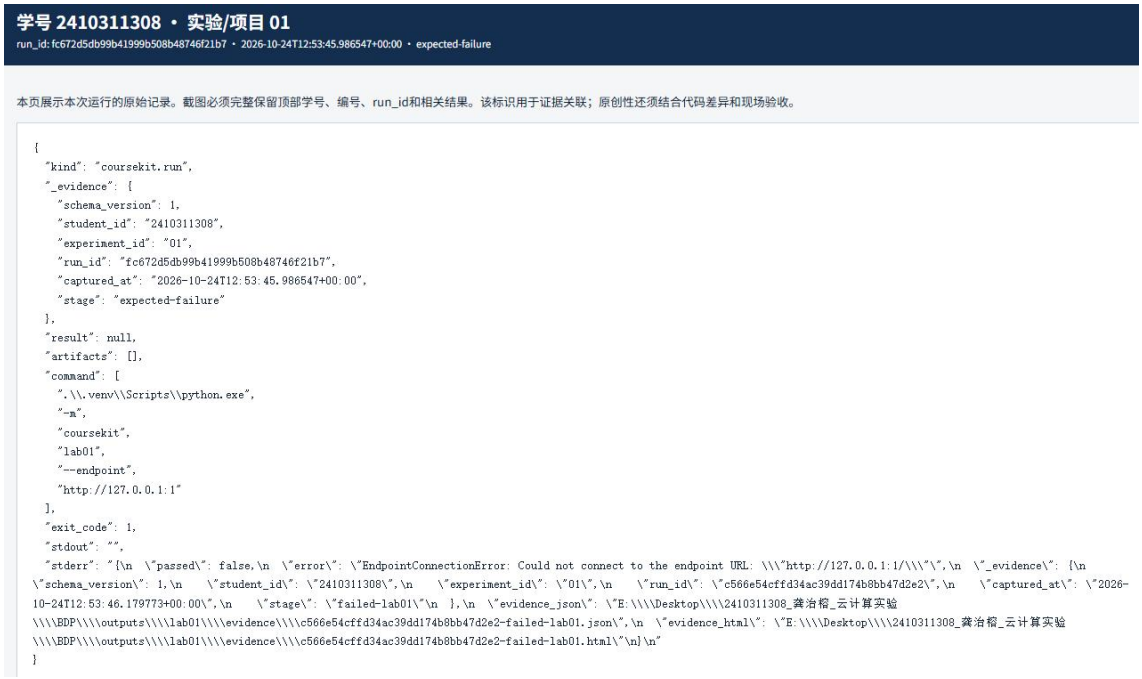
6. 故障注入与恢复

6.1 指向无效 endpoint

先确认本地端口 1 没有运行课程服务，再把 endpoint 明确指向 http://127.0.0.1:1。



退出码 1，EndpointConnectionError: Could not connect to the endpoint URL



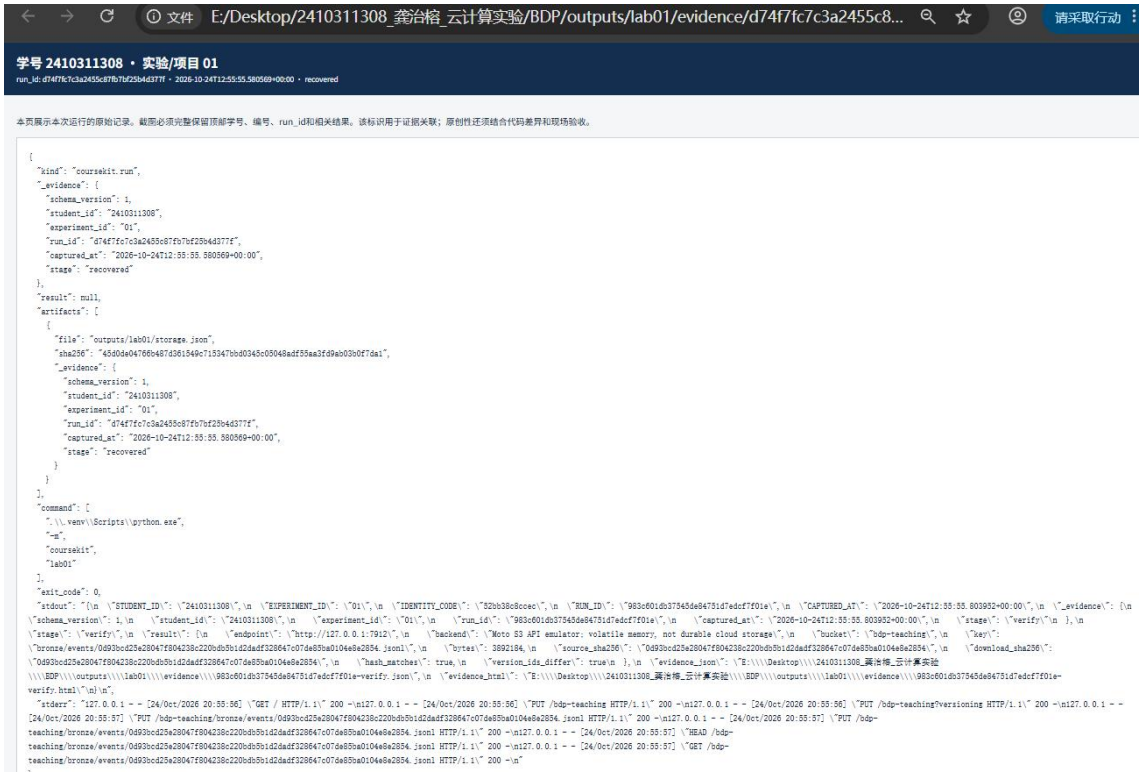
expected-failure 证据页：exit_code 1 与完整错误信息同屏，run_id fc672d5db99b41999b508b48746f21b7

6.2 恢复默认临时服务路径

去掉 --endpoint 参数后重新执行并留证，恢复运行是一次新的完整往返，不是回填旧记录。



recovered 运行 exit_code 0，内层 hash_matches 为 true



recovered 证据页，run_id d74f7fc7c3a2455c87fb7bf25b4d377f

7. 完成检查与下一章交接

依次执行最终身份登记、重新运行主实验与列出 data/raw 文件，确认原始数据、往返结果与个人改动均可用。

```
>> .\.venv\Scripts\python.exe -m coursekit identity --student-id $studentId --lab 01
 --stage final
{
  "STUDENT_ID": "2410311308",
  "EXPERIMENT_ID": "01",
  "IDENTITY_CODE": "52bb38c8ccec",
  "RUN_ID": "fe0288c57b864d88a093a74d29aec4e3",
  "CAPTURED_AT": "2026-10-24T12:57:18.149449+00:00",
  "_evidence": {
    "schema_version": 1,
    "student_id": "2410311308",
    "experiment_id": "01",
    "run_id": "fe0288c57b864d88a093a74d29aec4e3",
    "captured_at": "2026-10-24T12:57:18.149449+00:00",
    "stage": "final"
  },
  "stage": "final",
  "result": {
    "identity_note": "本人学号仅用于证据关联；每张评分截图必须带学号，不能替代原创性
验收。"
  },
  "evidence_json": "E:\\Desktop\\2410311308_龚治榕_云计算实验\\BDP\\outputs\\lab01\\
evidence\\fe0288c57b864d88a093a74d29aec4e3-final.json",
  "evidence_html": "E:\\Desktop\\2410311308_龚治榕_云计算实验\\BDP\\outputs\\lab01\\
evidence\\fe0288c57b864d88a093a74d29aec4e3-final.html"
}
PS E:\Desktop\2410311308_龚治榕_云计算实验\BDP> Get-ChildItem .\data\raw -Name
documents.jsonl
events.jsonl
questions.jsonl
```

data/raw 下有 documents.jsonl、events.jsonl、questions.jsonl，主实验再次通过

三、实验结果与分析

1. 环境与身份 Windows 10.0.26200，CPU，Python 3.12.10，解释器为 E:\Desktop\2410311308_龚治榕_云计算实验\BDP\.venv\Scripts\python.exe，pip check 通过。对照 requirements.lock.txt 逐包核对，99 个包全部安装且版本一致。学号 2410311308，实验编号 01，IDENTITY_CODE = 52bb38c8ccec；环境检查与最终结果两类截图均原生显示学号、实验号与 run_id。
2. 数据来源与组成数据由 coursekit seed --rows 20000 按 seed=42 生成，写入 data/raw/events.jsonl，共 20,102 行，由 20,000 条合法独特事件、100 条完全重复事件和 2 条不合法事件组成，文件 3,892,184 字节，SHA-256 为 0d93bcd25e28047f804238c220bdb5b1d2dadf328647c07de85ba0104e8e2854。独立脚本逐行复核与脚手架输出一致，未经手工删改。
3. 主实验往返结果
endpoint http://127.0.0.1:1839，桶 bdp-teaching，对象键 bronze/events/0d93bcd25e28047f804238c220bdb5b1d2dadf328647c07de85ba0104e8e2854.jsonl，原始与下载文件的 SHA-256 完全相同，字节数同为 3,892,184。hash_matches 与 version_ids_differ 均为 true，说明内容保持完整，同时同一键的两次写入确实产生了两个不同的 VersionId。两个判定能同时成立，是因为版本 ID 标识对象版本，内容摘要标识字节内容，两者用途不同。
4. 三次个人练习对照
基线运行对象键为 shared/raw/demo.json，SHA-256 为 23c0b20c65b4476c7e541044c09c88cd970bb02191681f804daef0b7ba49f0e8;改键后对象键变为 students/2410311308/raw/demo.json，摘要仍是同一个值；再把 payload 的 requests 由 3 改为 4，摘要变为 5657a58e4a955dcf741239aed81c7a600ca434c4b6c738557b8683d12b09d3f6。三次运行的 equal 判定都为 true。前两次键不同而摘要相同，是因为 payload 字节完全一样；第三次摘要改变，是因为字节内容变了。这组对照把对象地址与内容身份清楚地区分开来。
5. 哈希变式与语义一致性
除手册必做的一项预测外，另做了两项对照。同一份字节放到两个不同对象键下，摘要不变；只调整 JSON 缩进而不改数据值，摘要由 23c0b20c…f0e8 变为 de27ead6…a195，两个文件字节不同但解析后的字典相等。这说明字节一致性与业务含义一致性是两个不同层面的检查：摘要相同也不能证明事件含义正确，一条 input_tokens 为 -1 的记录，摘要同样可以稳定一致。
6. 故障定位与恢复
把 endpoint 指向 127.0.0.1:1 后，程序以退出码 1 结束并报 EndpointConnectionError: Could not connect to the endpoint URL。错误发生在建立 TCP 连接阶段，请求未到达任何 S3 服务，端口 1 为保留端口且无监听进程。因此这不是凭据错误，也不是数据质量问题，一个字节都未传输。去掉 --endpoint 参数后重新执行，得到新的 run_id 并再次通过双哈希核对。
7. 模拟与真实云的边界
本实验已验证：本地 HTTP 请求真实发出（访问日志出现 PUT、HEAD、GET）、S3 接口语义可用、字节层面内容一致。本实验未验证：事件数据业务合法（原始数据仍含 2 条不合法事件）、云存储耐久性、跨机容灾、真实访问控制与 IAM。Moto 默认服务停止后内存对象消失，没有副本与故障切换，测试凭据 testing 不含真实账号权限。因此本章完成的是本地云 API 模拟与数据往返校验，不能称为完成云部署。
8. 产物与限制

outputs/lab01 下生成 storage.json、storage.json.identity.json 与 downloaded-events.jsonl，evidence 目录保存各次运行的 JSON 与同名 HTML。个人副本 student_object_roundtrip.py 与改后代码在 submissions/lab01。限制：本次评价只有一个数据文件、一种对象存储实现，没有真实云账号、并发上传、分片下载和生命周期策略；单次运行的结果不能推广为生产环境的耐久性保证。

9. 运行记录索引

本次提交共 11 次真实运行的证据，均带学号 2410311308 与实验号 01，JSON 与同名 HTML 成对保存，可在证据仓库查看：

https://github.com/yiyun2026/bdp-coursework/lab01/evidence

start 55265e5e4a3140aeb926e097a13c7d45 身份登记

probe-01 79e5ce682cc248b8b49d15d463d979c9 解释器版本与路径

environment 2e11f1746eec4561a33b412e31208b30 依赖检查 pip check

probe-02 d13c3b0f0bee46df98bfad2fe0f47290 数据行数与首条事件

verify 983c601db37545de84751d7edcf7f01e 对象存储往返主实验

student_object_roundtrip 7ee816587ad44a0da9fa53a07c08c611 个人练习基线

student_object_roundtrip 28842a77ed8c4dbe95b930ac478fc2e3 改对象键后

student_object_roundtrip 8a31f29580b54ba58f2ce2efa19fefa5 改 payload 后

expected-failure fc672d5db99b41999b508b48746f21b7 无效 endpoint 连接失败

failed-lab01 c566e54cffd34ac39dd174b8bb47d2e2 失败的内层运行记录

recovered d74f7fc7c3a2455c87fb7bf25b4d377f 恢复运行并重新核对

四、实验理论回答

将相同数据保存为两个不同对象键，为什么不一定改变 SHA-256？

SHA-256 的计算输入只有对象 Body 的字节序列。桶名、对象键和 endpoint 属于 HTTP 请求的主机与路径部分，只用于定位对象，不进入摘要计算；服务端在下载时也不会对内容重新编码。只要字节相同，摘要必然相同，与它放在哪个键下无关。本实验基线键 shared/raw/demo.json 与改后键 students/2410311308/raw/demo.json 下摘要同为 23c0b20c…f0e8，正是这一推理的实测印证。

JSON 仅调整空格时，字节一致性与业务含义一致性是否相同？

不相同。仅调整缩进后，两个文件字节不同，SHA-256 也不同（23c0b20c…f0e8 变为 de27ead6…a195），但用 json 解析出的字典完全相等，业务值没有变化。字节一致性是对存储与传输的检查，业务含义一致性需要解析后按字段比较。评价数据是否被篡改要看摘要，评价数据处理是否出错要看字段值，二者不能互相替代。

上传一个 input_tokens=-1 的事件文件成功，能证明什么？不能证明什么？

能证明的是：客户端与 S3 接口之间的请求成功、对象写入成功、下载回来的字节与上传的字节一致，传输链路没有截断或改写。不能证明的是：这批数据业务上合法可用。对象存储按字节保存完整对象，不做字段校验，负的 Token 数与业务规则无关，因此接口成功与数据合法是两件事。本实验原始文件中确实存在一条 event_id 为 bad-negative、input_tokens 为 -1 的记录，它的往返同样通过，正好说明这一点。

为什么临时 Moto 服务停止后，本课程仍能开展实验 02？

实验 02 读取的是本地 data/raw/events.jsonl，而不是 Moto 中的对象。本机 Moto 服务退出后内存对象消失，它只是模拟 S3 接口、验证往返完整性的临时环境；跨实验保留的教学源是本地原始文件，因此实验 02 不依赖模拟接口继续运行，也不需要重新生成数据。

如果改用真实云对象存储，还需要验证哪些身份、权限、持久性与成本条件？

身份与权限方面，需要真实账号、访问密钥或角色授权，并验证最小权限策略、跨账号访问控制与对象加密是否生效；持久性方面，需要验证多副本或跨可用区冗余、版本管理与生命周期规则，并做真实故障演练而不是只看服务承诺；成本方面，需要记录存储容量、请求次数、网络出流量与版本保留带来的费用，并与预算告警核对。这些条件本实验均未覆盖，不能用本地模拟结果代替。

桶启用版本后，向同一键写入两次相同内容，为什么 hash_matches 与 version_ids_differ 可以同时为 true？

两者测量的是不同的东西。version_ids_differ 比较的是两次写入产生的 VersionId，它们标识的是两次对象版本，版本号不同是版本机制的正常表现；hash_matches 比较的是下载副本与原始文件的内容摘要，两次写入的字节完全一样，所以摘要相同。版本 ID 回答“这是第几次写入”，内容摘要回答“这几个字节是不是同一份”，二者并不矛盾。

对象键里包含斜杠，为什么不能把它当作文件系统目录？

对象存储的键是一整个字符串，斜杠只是键名中的一个普通字符，服务端不会据此建立层级目录，也不能原地修改对象中的某一行。桶与对象键共同定位一个完整对象，读取必须整体取出，更新必须整体覆盖。本实验用 students/2410311308/raw/demo.json 只是命名约定，便于不同提交者区分，真正的隔离仍要依赖访问控制，而不是键名。

五、AI 使用披露

本实验在课程允许的范围内使用了 AI 助手，用途限于三类：理解实验手册中命令参数的含义、分析报错信息、整理报告的行文结构。AI 没有参与任何一次实际运行，报告中出现的哈希、行数、run_id、endpoint 与判定结果均来自本机真实执行。

核对方法有三项。第一，每次截图前都打开证据 HTML，确认页面顶部原生显示的学号、实验号 01 与正文 run_id 一致，并把这段身份栏完整截入画面。第二，对主实验的 SHA-256 另写独立脚本，直接读取原始文件与下载副本重新计算，与程序输出比对。第三，把远端课程仓库与本地克隆做版本比对，确认所用手册与评分标准未被改动，避免依据过期文档操作。

本人独立完成的内容：核心哈希预测及其理由、make_key() 的修改方案与覆盖问题的解释、三项对照结果的解读、能力边界表逐项判断，以及本报告的全部运行取证。

六、实验心得

这次实验是我第一次真正把文件经 HTTP 接口上传再取回来核对。以前觉得“上传成功”就等于“数据没问题”，做完才发现接口返回 200 只是说请求被接受了，内容是否被改写要另算一遍 SHA-256 才能下结论。当看到原始与下载两行的摘要完全一致时，我才明白内容摘要这种看起来很枯燥的东西，实际上是判断数据有没有被动过的唯一可靠依据。

让我印象最深的是修改对象键的那一步。我原本以为换了存储位置，哈希总该变一下吧，结果摘要一模一样。回头想才通：哈希算的是内容，键只负责找东西。这个认识让我理解了为什么课程要用内容哈希来构造对象键——相同内容自然落到同一个键，天然去重，也天然避免覆盖。但老师强调它不替代访问控制，我也认可：键可控不等于权限可控，别人只要知道键名照样能写进来。

故障那一步也让我改了一个坏习惯。以前看到报错第一反应是怀疑数据或代码，这次错误信息写得很清楚是连接失败，端口 1 根本没有服务在听，请求连服务都没碰到。把连接失败错认成数据问题，会让排查方向完全跑偏。后来去掉参数重跑就恢复了，也让我意识到保留失败记录和恢复记录同样重要，只交成功结果说明不了问题被真正处理过。

最后是关于能力边界的部分。Moto 只是本机内存里的模拟服务，进程一停对象就没了。虽然它让我完整走通了上传下载的流程，但耐久性、跨机容灾、真实 IAM 这些一个都没验证

到。写报告时我特意把“已验证”和“未验证”分开列，不把模拟跑通说成云部署完成。

七、评语

指导教师：刘锦行